

FOUNDATIONS OF COMPUTER SCIENCE



Prof. Dr. Claudia Nuber
TH Degendorf, WS2025/2026



Week 1: Introduction

Overview & objectives, basic terminologies,
first look at automata problems



WELCOME & INTRODUCTION



- 37 years old
- Married, two step kids
- Mathematician
- Former Strategy Consultant (Oliver Wyman)
- Former Head of Corporate Planning at Krones AG in Neutraubling

WELCOME & INTRODUCTION

Thise course lays the theoretical and technical foundations of computer science that are essential for any serious work in Artificial Intelligence.

Understanding formal logic, computability, and digital circuits gives you the tools to reason about what machines can (and cannot) do, and how they work at the most fundamental level.

Why this course matters today



Digital logic powers AI

Algorithms & automata underpin modern machine learning; understanding architecture drives innovation



Ethical and societal perspective

Knowing the limits of computation fosters realistic expectations of AI in society (health, law, autonomous systems)



Sharpened and analytical thinking

Working through abstract models helps to develop precise, structured and critical thinking



COURSE TOPICS



Modul catalogue

- **Theory of computer science**
 - Logic
 - Computability
 - Finite automata
 - Formal languages
 - Complexity
- **Foundations of computer engineering**
 - Digital gates
 - Digital circuits
 - Computer architecture

WEEKLY COURSE PLAN

Chapter	Topic
1	Intro to Theoretical CS
2	Logic & Propositional Calculus
3	Proofs, Sets, Induction
4	Complexity Theory Basics
5	Automata & Regular Languages
6	Finite Automata – DFA & NFA
7	Formal Languages & Grammars
8	Turing Machines & Computability

Chapter	Topic
9	Digital Logic & Gates
10	Combinational Circuits
11	Sequential Circuits & FSM
12	Computer Architecture I
13	Computer Architecture II
14	Computer Architecture III
15	Review & Presentations

Key learning objectives are fixed. Which focus we have will depend on individual learning progress within this term



LEARNING OBJECTIVES



Understand key
theoretical concept in
computer science



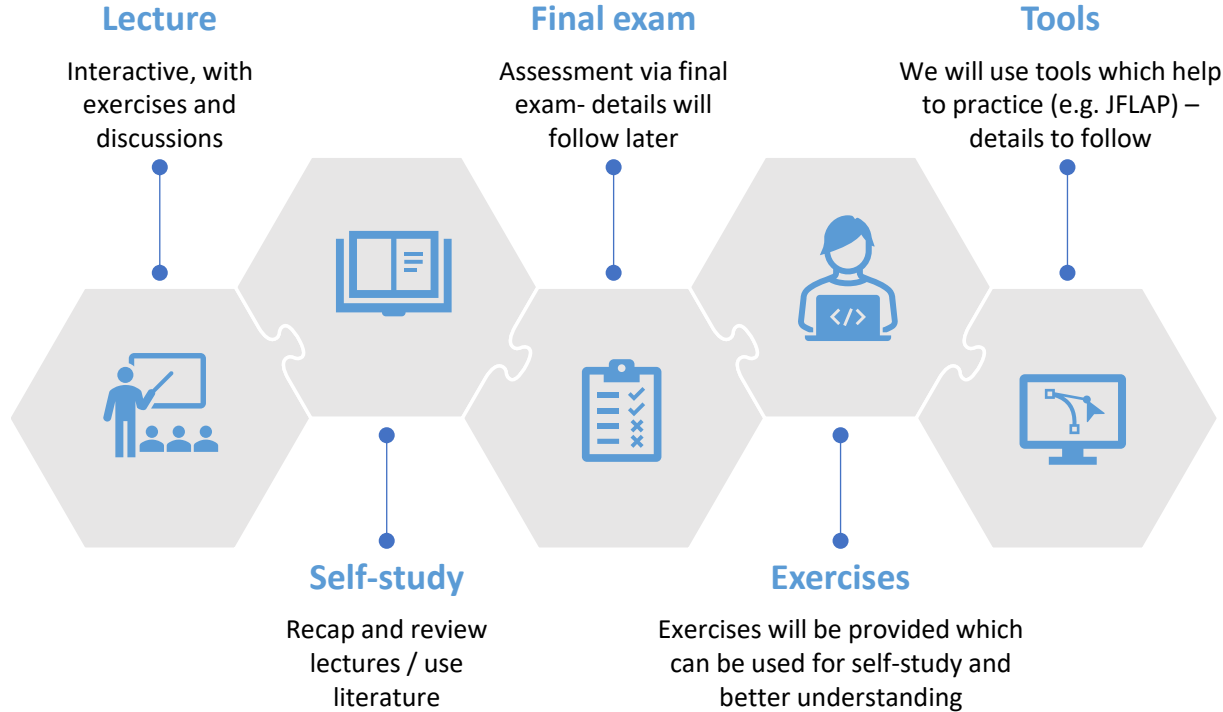
Gain practical skills in
digital logic and
architecture



Develop problem
solving and analytical
thinking



COURSE STRUCTURE



HOW DO WE WANT TO WORK TOGETHER



- **Be on time.** Arriving late disrupts the class.
- **Turn off your phone** or set it to silent mode. No texting or Social Media during the lecture.
- **Avoid private conversation** during class.
- **Ask questions** whenever something is unclear.
- **Participate actively.** Your engagement helps everyone learn
- **Stay for the entire session.** Avoid leaving early unless absolutely necessary.
- **Take notes.** It helps you stay focused and retain information.
- **Come prepared.** Read the assigned materials before class



QUESTIONS



Questions from your side on organization,
goals, way of working, ...?

LITERATURE OVERVIEW



- *Introduction to the Theory of Computation* — Michael Sipser
- *Digital Design and Computer Architecture* — David Harris & Sarah Harris
- *Automata and Computability* — Dexter Kozen
- *Computer Science: An Overview* — J. Glenn Brookshear & Dennis Brylow
- *JFLAP: An Interactive Formal Languages and Automata Package* — Susan H. Rodger & Thomas W. Finley
- *Digital Circuit Design (Lecture Script)* — Erich Hehner
- *Digital Design* — M. Morris Mano & Michael D. Ciletti
- *Computer Organization and Design* — David A. Patterson & John L. Hennessy
- *Fundamentals of Digital Logic* — Stephen Brown & Zvonko Vranesic



ONLINE RESOURCE FOR SELF-STUDY



1. Introduction to Computer Science

by Sedgewick & Wayne

<https://introcs.cs.princeton.edu>

A clear and interactive introduction to computer science fundamentals with Java examples and visualizations.

2. The Elements of Computing Systems (Nand2Tetris)

nand2tetris.org

Project-based learning resource that guides students through building a computer from basic logic gates to a working OS.

3. JFLAP Tutorials

jflap.org/tutorial

Official tutorials and examples for practicing automata, formal languages, and Turing machines interactively.

4. Digital Circuits & Computer Architecture (MIT OpenCourseWare)

<https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-004-computation-structures-spring-2017/>

Comprehensive video lectures and materials on digital logic design and computer architecture.



COMPUTER SCIENCE

“
The study of
computers and
how they can be
used

-- *Oxford Dictionary*
”

Is based on concepts like...

- Algorithms
- Abstraction
- Creativity
- Data
- Complexity
- Computability
- Automata



BASIC CONCEPTS OF COMPUTER SCIENCE

Algorithms

“What is a clear, step-by-step method to solve this problem?”

A set of steps that define how a task is performed (e.g. cooking, directions, manuals, ...). A finite series of well-defined instructions.

Program

“How can I tell a computer what to do to solve a problem?”

A representation of an algorithm is a program. It needs to be represented in a form that is compatible with the machine.

Abstraction

“What details can I ignore so I can focus on what really matters?”

Simplifying complex systems by hiding unnecessary detail and focusing on high-level ideas. We manage complex systems by ignoring the internal details and focus on interaction on the same level.

Creativity

“How can I come up with a new or better way to solve this?”

Discovering and applying new algorithms to solve problem in the world. New modes of digital expression.

Data

“What information do I need to make a decision or find a pattern?”

Collecting, organizing, analyzing and interpreting facts and figures (digitally represented information) to support reasoning or conclusions, search for patterns, create simulations, correlate connections

Complexity

“What makes some problems computationally hard and others easy?”

Problems can be classified according to their computational complexity (e.g. sorting problems are easy, cryptography is hard) – different approach necessary depending on the kind of problem

Computability

“What problems can be solved by a computer or algorithm and what cannot be solved?”

Certain basic problems cannot be solved by computers (e.g. predicting the outcome of any conversation, 100% spam detection)

Automata

“How can machine models solve problems depending on their memory limitations & structure?”

What can machine models compute, how do we design it and how to verify correctness. Automata deals with the definitions and properties of mathematical models of computation.



RECAP: COMPUTER SCIENCE

“
The study of
computers and
how they can be
used

-- *Oxford Dictionary*

”

Is based on concepts like...

- Algorithms
- Abstraction
- Creativity
- Data
- Complexity
- Computability
- Automata



RECAP: BASIC CONCEPTS OF COMPUTER SCIENCE

Algorithms

“What is a clear, step-by-step method to solve this problem?”

A set of steps that define how a task is performed (e.g. cooking, directions, manuals, ...). A finite series of well-defined instructions.

Program

“How can I tell a computer what to do to solve a problem?”

A representation of an algorithm is a program. It needs to be represented in a form that is compatible with the machine.

Abstraction

“What details can I ignore so I can focus on what really matters?”

Simplifying complex systems by hiding unnecessary detail and focusing on high-level ideas. We manage complex systems by ignoring the internal details and focus on interaction on the same level.

Creativity

“How can I come up with a new or better way to solve this?”

Discovering and applying new algorithms to solve problem in the world. New modes of digital expression.

Data

“What information do I need to make a decision or find a pattern?”

Collecting, organizing, analyzing and interpreting facts and figures (digitally represented information) to support reasoning or conclusions, search for patterns, create simulations, correlate connections

Complexity

“What makes some problems computationally hard and others easy?”

Problems can be classified according to their computational complexity (e.g. sorting problems are easy, cryptography is hard) – different approach necessary depending on the kind of problem

Computability

“What problems can be solved by a computer or algorithm and what cannot be solved?”

Certain basic problems cannot be solved by computers (e.g. predicting the outcome of any conversation, 100% spam detection)

Automata

“How can machine models solve problems depending on their memory limitations & structure?”

What can machine models compute, how do we design it and how to verify correctness. Automata deals with the definitions and properties of mathematical models of computation.



EXAMPLE: TYPES OF AUTOMATA

	<u>Memory</u>	<u>Power</u>	<u>Example Problem</u>	<u>Exemplary Use Case</u>
Finite Automata	No extra memory beyond current state	Recognizes regular language	Check if a binary string ends in "01": - Input: "1101" → Accept - Input: "1000" → Reject	Search functionalities (e.g. in a text)
Pushdown Automata	Stack (LIFO – Last in, First Out)	Recognized context-free languages	Check if parentheses are balanced: - Input: "(()())" → Accept - Input: "(()" → Reject	Syntax parsing (e.g. mathematical expression, HTML Tag, ...)
Turing Machines	Infinite tape (unbounded memory)	Recognizes computable languages (decidable and semi-decidable)	Add 1 to a binary number Input: "1011" → Output: "1100" Or: Will a program halt or run forever? (Halting Problem)	General purpose computation (anything a modern computer can do) or studying limites of computation



STRINGS AND LANGUAGES

- An **alphabet** is a nonempty finite set
 - The members of the alphabet are the **symbols** of the alphabet
 - We use capital Greek letters Σ and Γ to designate alphabets
-
- A **string** over an alphabet is a finite sequence of symbols from that alphabet, usually written next to one another and not separated by commas

Examples

$$\Sigma = \{0, 1\}$$

$$\Sigma_2 = \{0, 1, a, b, 3, z\}$$

$$\Gamma = \{a, b, c, d, e, f\}$$

$$\Sigma = \{0, 1\}$$

→ 01001 is a string

$$\Gamma = \{a, b, c, d, e, f\}$$

→ affe is a string



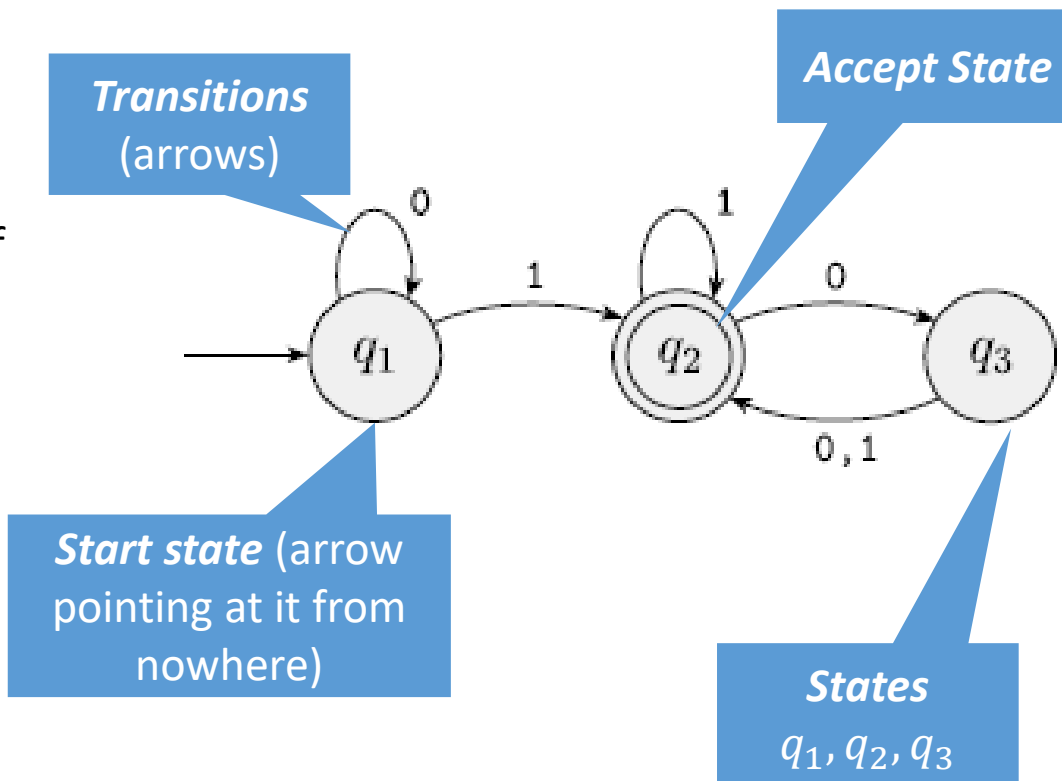
STRINGS AND LANGUAGES

- If w is a string over Σ , the **length** is the number of symbols that it contains, written $|w|$
- The string of length zero is called the **empty string** written as ε
- If w has length n we can write $w = w_1 w_2 \dots w_n$ where each $w_i \in \Sigma$
- The **reverse** of w is often written as w^R and is obtained by writing w in the opposite order
- String z is a **substring** of w if it appears consecutively within w
- If we have a string x and a string y the **concatenation** of x and y is written in xy and obtained by appending y to the end of x
- A **language** is a set of strings

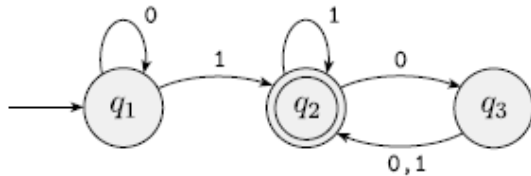


STATE DIAGRAMS

To describe the theory of finite automata, we use a so-called **state diagram**, see example on the right



STATE DIAGRAMS



1101

$q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_3 \rightarrow q_2$: Accept

110

$q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_3$: Reject

BITS AND BINARY NUMBERS

- Signals in most electronical systems use just two different discrete values and are therefore said to be **binary**. A binary digit, called a **bit**, has two values: 0 and 1.
- How a pattern of bits is interpreted as a number depends on the code system in which it resides, e.g. $(0111)_2$ indicates we are in the **binary** system, $(0111)_{10}$ that we are in the **decimal** system
- The most important numeral systems use a base of sixteen (**Hexadecimal**), base two (**Binary**), base 10 (**Decimal**). Hexadecimal numbers are sometimes noted with the **prefix 0x** (e.g. **0xEF20**).

Hex.	Dual				Dec
0	0	0	0	0	00
1	0	0	0	1	01
2	0	0	1	0	02
3	0	0	1	1	03
4	0	1	0	0	04
5	0	1	0	1	05
6	0	1	1	0	06
7	0	1	1	1	07
8	1	0	0	0	08
9	1	0	0	1	09
A	1	0	1	0	10
B	1	0	1	1	11
C	1	1	0	0	12
D	1	1	0	1	13
E	1	1	1	0	14
F	1	1	1	1	15

BITS AND BINARY NUMBERS

Example: Decimal number 255

Base 10:

$$255 = 2 \times 100 + 5 \times 10 + 5 \times 1 = 2 \times 10^2 + 5 \times 10^1 + 5 \times 10^0$$

Base 16:

$$255 = 15 \times 16^1 + 15 \times 16^0 = 240 + 15 = FF$$

Base 2:

$$\begin{aligned} 255 &= 1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = \\ &= 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 11111111 \end{aligned}$$

$$(255)_{10} = (11111111)_2 = (FF)_{16}$$



CONVERSION RULES

Number in base r to decimal

Expand the number in a power series and add all terms.

Example: $(011)_2$
(Base 2 to Base 10 = decimal)

$$(011)_2 = 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 3$$

Number in decimal to base r

Divide number and all successive quotients by r and accumulating the remainders. This continues until the quotient becomes 0

Example: $(11)_{10}$ (Base 10 to Base 2)

11	$\div 2$	= 5	Remainder: 1
5	$\div 2$	= 2	Remainder: 1
2	$\div 2$	= 1	Remainder: 0
1	$\div 2$	= 1	Remainder: 1

$(1011)_2$

EXERCISE

Example: Binary number 10101100

Base 2:

$$172 = 1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = \\ = 128 + 32 + 8 + 4 = 10101100$$

Base 10:

$$172 = 1 \times 100 + 7 \times 10 + 2 \times 1 = 1 \times 10^2 + 7 \times 10^1 + 2 \times 10^0$$

Base 16:

$$172 = 10 \times 16^1 + 12 \times 16^0 = 160 + 12 = AC$$

$$(10101100)_2 = (172)_{10} = (AC)_{16}$$



PERFORMING OPERATIONS ON INFORMATION

Now we know:

- Each bit is either 0 or 1
- Information is stored via combination of bits / binary variables are used to represent information



But: **How can we actually process binary numbers?** How can we perform operations on these (e.g. addition, comparison, etc.)?

“Binary numbers are the language, Logic gates are the grammar.”



LOGIC GATES

- **Logic gates** are digital circuits that take one or more binary inputs and produce a binary output
- Logic gates are drawn with a symbol showing the input(s) and the output. Inputs are usually drawn on the left (or top) and outputs on the right (or bottom).
- Typically letters at the beginning of the alphabet (e.g. “A”) are used for inputs, and “Y” for the gate output
- The relationship between the inputs and outputs can be described with a truth table or a Boolean equation
- A **truth table** lists inputs on the left and the corresponding outputs on the right in a usual table format – one row for each possible combination of inputs
- A **Boolean equation** is a mathematical expression using binary variables

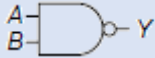
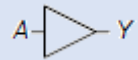


LOGIC GATES (1/2)

Gate	Symbol	Notation	Boolean Equation	Truth Table	Example															
NOT		$\neg A$	$Y = \bar{A}$	<table><tr><th>A</th><th>Y</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	Y	0	1	1	0	$A=1 \rightarrow Y=0$									
A	Y																			
0	1																			
1	0																			
AND		$A \wedge B$	$Y = A \cdot B$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	0	1	0	0	1	1	1	$A=1, B=1 \rightarrow Y=1$
A	B	Y																		
0	0	0																		
0	1	0																		
1	0	0																		
1	1	1																		
OR		$A \vee B$	$Y = A + B$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	1	$A=1, B=0 \rightarrow Y=1$
A	B	Y																		
0	0	0																		
0	1	1																		
1	0	1																		
1	1	1																		
XOR (exactly one input true)		$A \oplus B$	$Y = A \oplus B$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	0	$A=1, B=0 \rightarrow Y=1$
A	B	Y																		
0	0	0																		
0	1	1																		
1	0	1																		
1	1	0																		



LOGIC GATES (2/2)

Gate	Symbol	Notation	Boolean Equation	Truth Table	Example															
NAND <i>(true unless both are true)</i>		$\neg(A \wedge B)$	$Y = \overline{A \cdot B}$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	1	1	0	1	1	1	0	A=1, B=1 $\rightarrow$ Y=0
A	B	Y																		
0	0	1																		
0	1	1																		
1	0	1																		
1	1	0																		
NOR <i>(true only if both are false)</i>		$\neg(A \vee B)$	$Y = \overline{A + B}$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	0	1	0	0	1	1	0	A=0, B=0 $\rightarrow$ Y=1
A	B	Y																		
0	0	1																		
0	1	0																		
1	0	0																		
1	1	0																		
BUFFER		$A = B$	$Y = A$	<table><tr><th>A</th><th>Y</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	A	Y	0	0	1	1	A=0 $\rightarrow$ Y=0									
A	Y																			
0	0																			
1	1																			



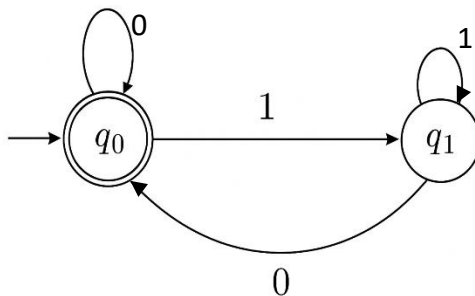
LET'S PUT THINGS TOGETHER

How can we model simple machines – like checking if a binary number is divisible by 2?



Language $L = \{w \mid w \text{ is a binary number divisible by } 2\} = \{w \mid w \text{ is a binary string ending with } 0\}$

State diagram:



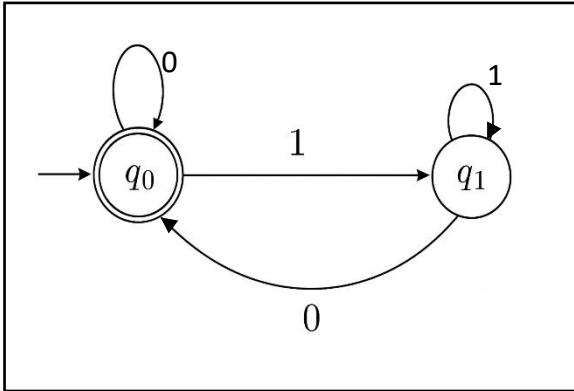
q_0 = Start and accept state

q_1 = Non-accept state

Input alphabet: $\{0,1\}$

- Transitions depend on the last input bit
- Accepting state says “this word belongs to the language”

LET'S PUT THINGS TOGETHER



Input	Decimal number	State sequence	Accepted? / Belongs to L ?
0	0	$q_0 \rightarrow q_0$	Yes
10	2	$q_0 \rightarrow q_1 \rightarrow q_0$	Yes
11	3	$q_0 \rightarrow q_1 \rightarrow q_1$	No
1010	10	$q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_0$	Yes
1111	15	$q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_1 \rightarrow q_1$	No

LET'S PUT THINGS TOGETHER

Logic Gates:

Input:

A = Last bit of the binary input (0 or 1)

Output:

Y = 1 (Yes) in case input is 0, 0 (No) in case input is 1

Boolean expression:

$$Y = \bar{A}$$

Symbol:



Truth table:

A	Y
0	1
1	0

For a binary string

$$w = b_n b_{n-1} \dots b_1 b_0$$

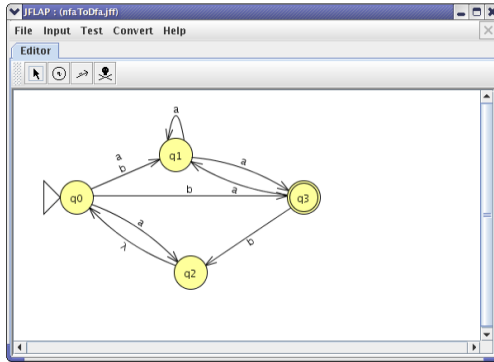
we take the last bit b_0

and take $A = b_0$ so that

$$Y = \bar{A} = \bar{b}_0$$

JFLAP

JFLAP is a software for experimenting with formal language topics. It is a package of graphical tools which can be used as an aid in learning the basic concepts of Formal Languages and Automata Theory.



Please visit

<https://www.jflap.org>

- Install & start JFLAP for later usage in this course until next week
- Try to model one of the state diagrams of this course in JFLAP & upload to iLearn until 13 October eod

NOTES FOR LAUNCHING JFLAP

- Make sure that Java is installed on your computer
 - JFLAP needs JRE or JDK
 - Download possible e.g. here: <https://adoptium.net/>
 - JFLAP can then also be run under Mac or Linux (e.g. right click → Open with Java Launcher)
 - Direct download link to JFLAP:
<http://www.jflap.org/jflaptmp/jflap.jar>
- Save an launch with Java Environment



THANK YOU
FOR YOUR ATTENTION

